

NLP Assignment-2

MT20205015 Aditya Pareek

Q1-A)

Parse Transition Table

```
[1] !pip install -q spacy
✓ 21s !python -m spacy download en_core_web_sm

Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
    12.8/12.8 MB 78.3 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
▲ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.
```

→ install the spacy library and download the english language model required for performing NLP tasks like tokenization and dep. parsing.

```
import spacy
from collections import deque

nlp = spacy.load("en_core_web_sm")

sentence = "I parsed this sentence correctly"
doc = nlp(sentence)

stack = ["ROOT"]
buffer = deque(range(len(doc)))
steps = []

def record(action, dep=""):
    steps.append((
        [doc[i].text if i != "ROOT" else "ROOT" for i in stack],
        [doc[i].text for i in buffer],
        action,
        dep
    ))
```

```

record("Initial")

while buffer or len(stack) > 1:

    if len(stack) >= 2 and stack[-1] != "ROOT" and stack[-2] != "ROOT":

        top = doc[stack[-1]]
        second = doc[stack[-2]]

        if second.head == top:
            stack.pop(-2)
            record("LEFT-ARC", f"{top.text} → {second.text}")
            continue

        if top.head == second:
            stack.pop()
            record("RIGHT-ARC", f"{second.text} → {top.text}")
            continue

    if buffer:
        stack.append(buffer.popleft())
        record("SHIFT")
    else:
        break

print(f"{'STACK':<40} {'BUFFER':<60} {'TRANSITION':<15} DEPENDENCY")
print("-" * 110)

for stack_s, buffer_s, action, dep in steps:
    print(f"{'str(stack_s):<40'} {'str(buffer_s):<60'} {'action:<15'} {'dep:}'")

```

STACK	BUFFER	TRANSITION	DEPENDENCY
['ROOT']	['I', 'parsed', 'this', 'sentence', 'correctly']	Initial	
['ROOT', 'I']	['parsed', 'this', 'sentence', 'correctly']	SHIFT	
['ROOT', 'I', 'parsed']	['this', 'sentence', 'correctly']	SHIFT	
['ROOT', 'parsed']	['this', 'sentence', 'correctly']	LEFT-ARC	parsed → I
['ROOT', 'parsed', 'this']	['sentence', 'correctly']	SHIFT	
['ROOT', 'parsed', 'this', 'sentence']	['correctly']	SHIFT	
['ROOT', 'parsed', 'sentence']	['correctly']	LEFT-ARC	sentence → this
['ROOT', 'parsed']	['correctly']	RIGHT-ARC	parsed → sentence
['ROOT', 'parsed', 'correctly']	[]	SHIFT	
['ROOT', 'parsed']	[]	RIGHT-ARC	parsed → correctly

Implement a trans. based dependency parser using spaCy that process a sentence by applying **SHIFT**, **LEFT-ARC**, and **RIGHT-ARC** operations while recording the stack buffer and dependency relation at each step.